

Modélisez une infrastructure dans le cloud

CONTEXTE

InduTechData est une entreprise de taille moyenne spécialisée dans l'analyse de données industrielles depuis 15 ans.

Son système d'information actuel repose sur une infrastructure **on-premise**, composée notamment de :

- un cluster SQL Server contenant 40 To de données critiques (ERP, CRM),
- une baie SAN pour les données non structurées (logs, fichiers, IoT),
- un Active Directory pour la gestion des identités,
- des applications métiers internes (ERP, CRM).

Au sein de notre entreprise, InduTechData, nous faisons face à une croissance rapide des données issues des solutions IoT, avec des flux en temps réel que notre infrastructure on-premise actuelle ne peut plus absorber efficacement.

Dans ce contexte, nous avons engagé une réflexion pour moderniser notre architecture en intégrant des services cloud, afin d'améliorer la scalabilité, les performances et la capacité de traitement des données en temps réel.

Toutefois, nous souhaitons conserver notre système d'information existant, notamment nos bases SQL Server, notre Active Directory ainsi que nos applications métiers (ERP/CRM), qui restent au cœur de nos opérations.

SI ON PREMISE



Active Directory



SQL Server cluster (40 To)
(CDC activé)



ERP / CRM



SAN 10 To (logs, fichiers,
données capteurs)



Sources IoT / capteurs

On-premise réseau
Customer Gateway .
tunnel VPN (IPsec:
TLS)

Internet

Virtual Private
Gateway (AWS)
VPC AWS

Active Directory (on-prem) → AWS Directory Service
Synchronisation identités / SSO

Connexion sécurisée (VPN
Site-to-Site / Direct
Connect)



IDENTITÉ

AD → AWS Directory Service

DATA BATCH

ERP → SQL → DMS → S3 → Redshift

DATA TEMPS RÉEL

IoT → Redpanda → S3 → Redshift

CLOUD AWS

VPC AWS

VPC

aws active directory

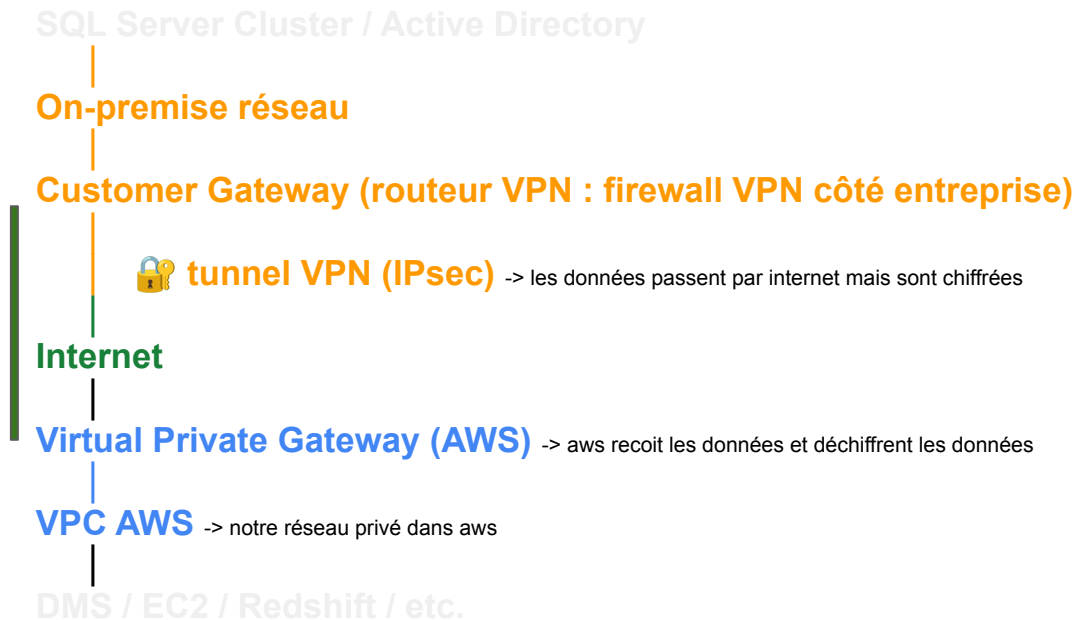


Real time data processing
avec redpanda

Redpanda Cloud

Le réseau

Le VPN relie le réseau on-premise à un VPC AWS via une Virtual Private Gateway côté AWS et un Customer Gateway côté entreprise.



Scalabilité des composants

L'architecture est scalable par couche.

Côté on-premise, les composants critiques comme Active Directory, SQL Server et le SAN peuvent être renforcés par réplication, clustering et ajout de capacité.

La connexion hybride peut évoluer d'un VPN Site-to-Site vers AWS Direct Connect pour obtenir plus de débit et de stabilité.

Côté AWS, les services managés permettent de scaler plus facilement : S3 scale automatiquement, DMS peut augmenter sa capacité de réplication, Redshift peut ajouter des nœuds ou passer en Serverless, et Redpanda Cloud peut augmenter le nombre de brokers et de partitions pour absorber plus de flux temps réel.

L'objectif est d'éviter les goulots d'étranglement et de permettre à chaque couche — identité, réseau, stockage, batch et temps réel — d'évoluer indépendamment.

Principe du moindre privilège (Least Privilege)

Le principe du moindre privilège consiste à accorder à chaque utilisateur, application ou service **uniquement les permissions strictement nécessaires** à sa mission, et rien de plus.

Dans AWS, cela se traduit par l'utilisation de **rôles IAM dédiés** avec des droits limités à un périmètre précis.

Exemples :

- une application lisant S3 ne dispose que des droits de lecture (`GetObject`)
- AWS DMS peut lire la source et écrire la cible, sans accès d'administration
- un analyste peut consulter Redshift sans modifier l'infrastructure

Bénéfices :

- réduction de la surface d'attaque
- limitation des erreurs humaines
- réduction de l'impact en cas de compromission
- meilleur contrôle des accès cloud

En cloud, la sécurité ne repose pas uniquement sur le réseau ou le chiffrement, mais aussi sur une gestion stricte des permissions.

Documentation : Composants cloud sélectionnés

Avantages

- architecture scalable
- découplage via S3
- hybrid → pas de rupture SI
- temps réel + batch gérés
- SSO unifié

Limitations

- complexité (plusieurs services)
- dépendance réseau (VPN)
- gestion coûts AWS
- Redpanda à maintenir

Points d'attention

- monitoring (CloudWatch)
- gestion des erreurs DMS
- gouvernance des données
- qualité des données (DQ)
- latence VPN

https://docs.google.com/document/d/1YjDe3FRBzcCYurSelh_Z-amCPwNWCT4Eh_T6eHkM6A8/edit?tab=t.0

Contexte

Simuler un système de tickets clients en temps réel, avec ingestion via Redpanda et traitement avec PySpark.

Producer Python → **Redpanda** → **Spark Streaming** → **Outputs (Parquet + JSON)**



- Producer = génère des tickets
- Redpanda = broker temps réel
- Spark = traitement streaming
- Output = stockage + analyse

Arborescence du projet final

```
redpanda_project_dockerized/

- .dockerignore
- create_topic.sh
- docker-compose.yml
- Dockerfile.producer
- Dockerfile.redpanda
- Dockerfile.spark
- producer_tickets.py
- README.md
- requirements_producer.txt
- run_spark.sh
- spark_redpanda_analysis.py
- wait_for_redpanda.sh
- output/  
checkpoints/

```

Commandes utiles

Montrer les services

`docker compose ps`

Montrer le producer

`docker compose logs producer`

Montrer Spark

`docker compose logs spark`

Montrer les outputs

`Get-Content .\output\tickets_by_type\batch_20\*.json`

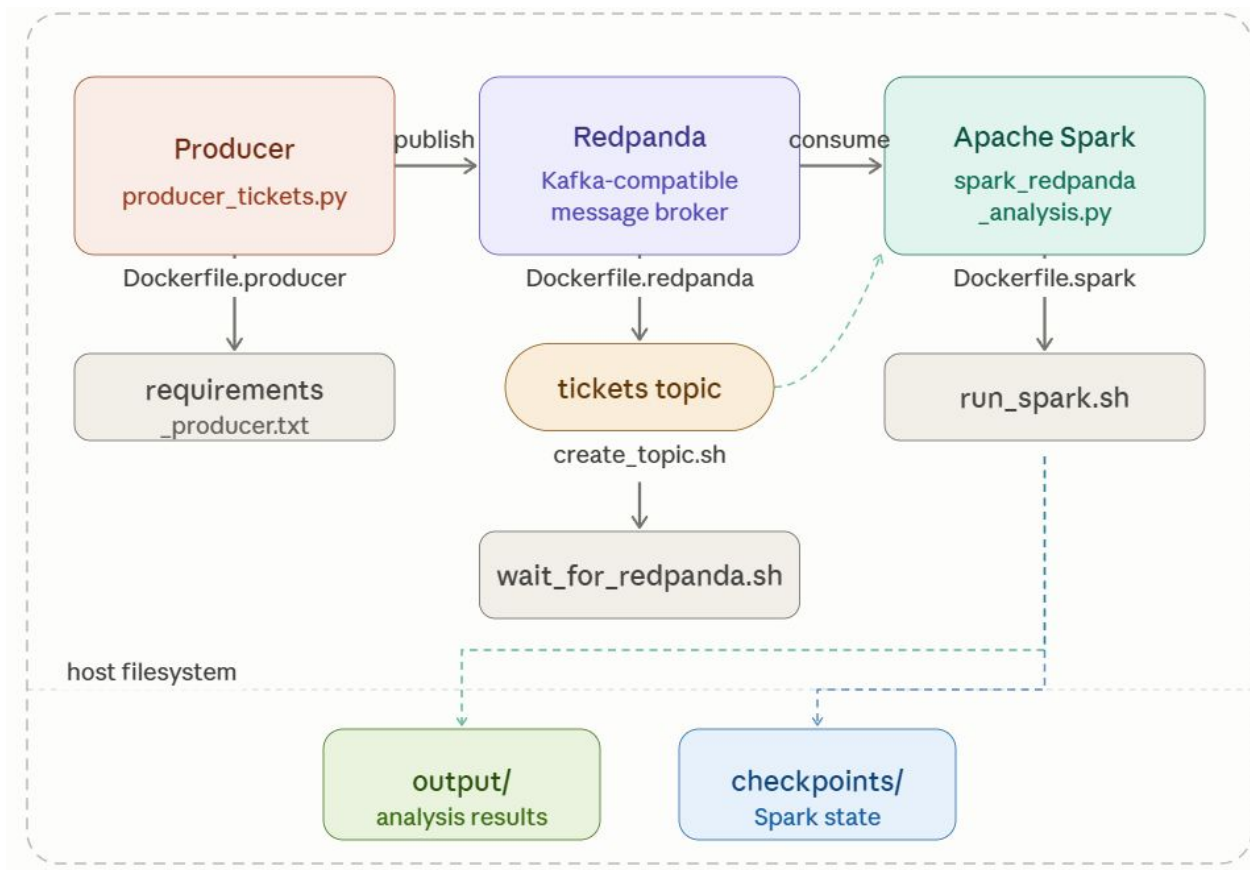
`Get-Content .\output\tickets_by_priority\batch_20\*.json`

`Get-Content .\output\tickets_by_team\batch_20\*.json`

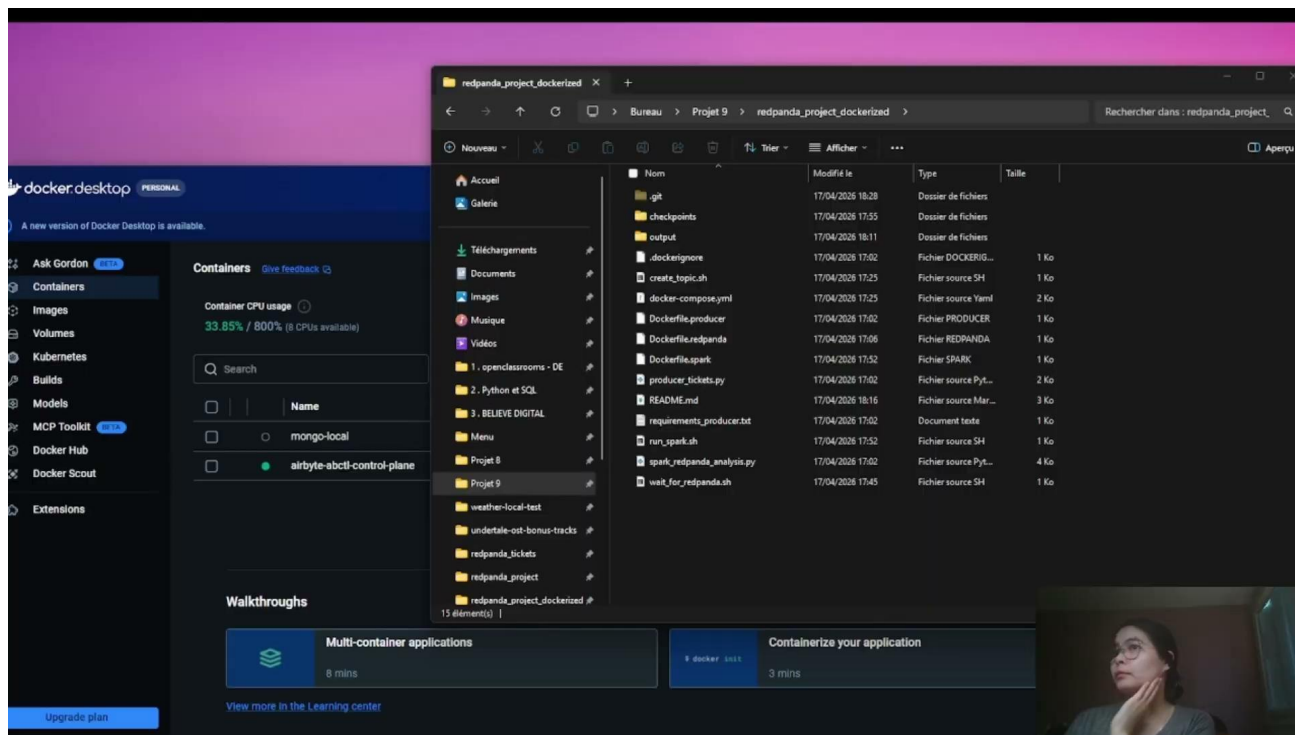
Et aussi :

`Get-ChildItem .\output\enriched_tickets`

Fonctionnement



Video de présentation



<https://github.com/ngocthaoly17/redpanda-streaming-pipeline/blob/main/README.md>

Annexe

https://docs.google.com/document/d/1YjDe3FRBzcCYurSelh_Z-amCPwNWCt4Eh_T6eHkM6A8/edit?tab=t.0